Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド）」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**）、データの処理（**Model**）、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド)」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**)、データの処理（**Model**)、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド)」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**）、データの処理（**Model**）、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド）」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**）、データの処理（**Model**）、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

Spring の特徴

1. DI(依存性注入)

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ(メソッド)」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC(Model-View-Controller)モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示(**View**)、データの処理(**Model**)、そしてユーザの入力と画面表示の制御(**Controller**)といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

長い段落の継続と文字欠落の検証

長文001Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文002Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文003Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文004Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文005Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文006Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文007Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文008Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文009Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文010Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文011Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文012Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。長文013Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率

よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。 長文014Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。 長文015Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。 長文016Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。 長文017Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。 長文018Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。 長文019Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。 長文020Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

表60行の改ページ検証

要素	JDBC	Spring Data JPA
R001 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R001E
R002 ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R002E
R003 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R003E
R004 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R004E
R005 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R005E
R006 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R006E
R007 ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R007E
R008 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R008E
R009 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R009E

表60行の改ページ検証

要素	JDBC	Spring Data JPA
R010 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R010E
R011 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R011E
R012 ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R012E
R013 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R013E
R014 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R014E
R015 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R015E
R016 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R016E
R017 ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R017E
R018 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R018E

表60行の改ページ検証

要素	JDBC	Spring Data JPA
R019 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R019E
R020 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R020E
R021 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R021E
R022 ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R022E
R023 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R023E
R024 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R024E
R025 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R025E
R026 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R026E
R027 ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R027E

表60行の改ページ検証

要素	JDBC	Spring Data JPA
R028 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R028E
R029 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R029E
R030 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R030E
R031 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R031E
R032 ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R032E
R033 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R033E
R034 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R034E
R035 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R035E
R036 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R036E

表60行の改ページ検証

要素	JDBC	Spring Data JPA
R037 ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R037E
R038 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R038E
R039 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R039E
R040 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R040E
R041 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar）を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R041E
R042 ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R042E
R043 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R043E
R044 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R044E
R045 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R045E

表60行の改ページ検証

要素	JDBC	Spring Data JPA
R046 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R046E
R047 ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R047E
R048 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R048E
R049 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R049E
R050 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R050E
R051 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R051E
R052 ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R052E
R053 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R053E
R054 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R054E

表60行の改ページ検証

要素	JDBC	Spring Data JPA
R055 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R055E
R056 外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。 R056E
R057 ハードコーディング	接続情報(URL、ユーザー名、パスワード)がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル(application.properties)に記載するため、セキュリティが向上し、環境に応じた変更も容易。 R057E
R058 冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。 R058E
R059 例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。 R059E
R060 リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。 R060E

Q001 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q002 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q003 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Q004 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q005 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q006 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Q007 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q008 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q009 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Q010 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q011 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q012 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Q013 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q014 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q015 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Q016 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q017 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q018 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Q019 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q020 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q021 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Q022 a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

Q023 b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

Q024 c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。

A クラスは渡されたリモコンの「スイッチ（メソッド）」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

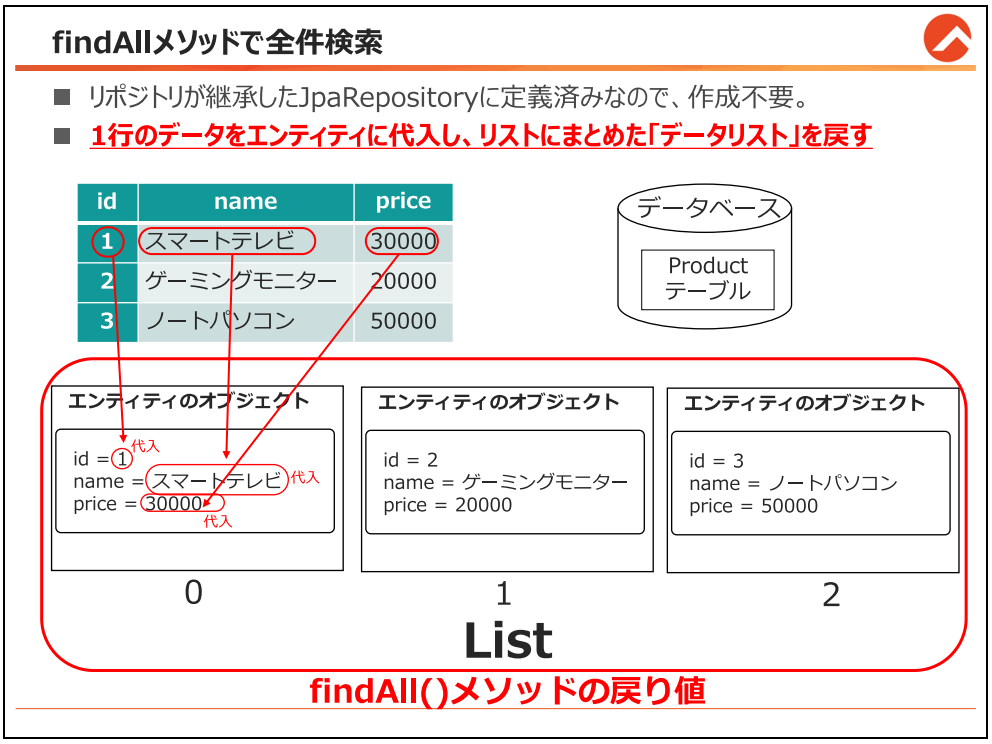
Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**）、データの処理（**Model**）、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。



図の説明とスライドを同じページに配置する検証